



Deep Learning

Tutorial 8B: Object Detection using Faster RCNN (Pre-Trained Model)

Submitted By: **Muhammad Mubashar**

Submitted To: **Dr Shahbaz Khan**

Reg No # **000000494171**

Semester: **4th**

Department of Robotics & Artificial Intelligence

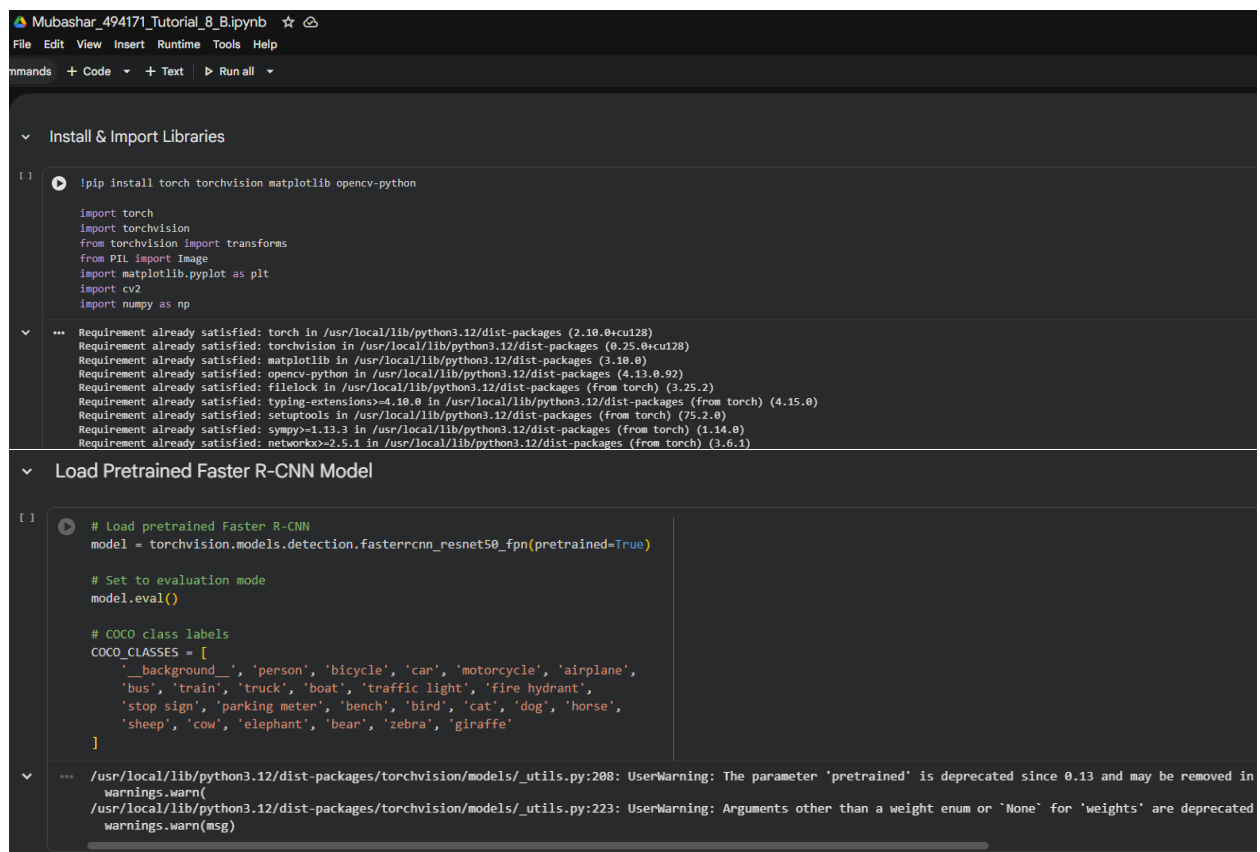
School of Mechanical & Manufacturing Engineering

NUST

Tasks:

- Test it with your own image
- Take another two models with different backbones
- Take a bunch of data for evaluating and then find its mAP

Implementation of these tasks are in *PyTorch*:



```
Mubashar_494171_Tutorial_8_B.ipynb ☆
File Edit View Insert Runtime Tools Help
In 1:
!pip install torch torchvision matplotlib opencv-python

import torch
import torchvision
from torchvision import transforms
from PIL import Image
import matplotlib.pyplot as plt
import cv2
import numpy as np

Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cu128)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.25.0+cu128)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.25.2)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)

In 2:
# Load pretrained Faster R-CNN
model = torchvision.models.detection.fasterrcnn_resnet50_fpn(pretrained=True)

# Set to evaluation mode
model.eval()

# COCO class labels
COCO_CLASSES = [
    '_background_', 'person', 'bicycle', 'car', 'motorcycle', 'airplane',
    'bus', 'train', 'truck', 'boat', 'traffic light', 'fire hydrant',
    'stop sign', 'parking meter', 'bench', 'bird', 'cat', 'dog', 'horse',
    'sheep', 'cow', 'elephant', 'bear', 'zebra', 'giraffe'
]

/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated
warnings.warn(msg)
```

Image Preprocessing Function

```
[1] def load_image(img_path):  
    image = Image.open(img_path).convert("RGB")  
  
    transform = transforms.Compose([  
        transforms.ToTensor()  
    ])  
  
    return image, transform(image)
```

Object Detection Function

```
[1] def detect_objects(model, image_tensor, threshold=0.5):  
    with torch.no_grad():  
        predictions = model([image_tensor])[0]  
  
    boxes = predictions['boxes']  
    labels = predictions['labels']  
    scores = predictions['scores']  
  
    filtered_boxes = []  
    filtered_labels = []  
    filtered_scores = []  
  
    for i in range(len(scores)):  
        if scores[i] > threshold:  
            filtered_boxes.append(boxes[i].numpy())  
            filtered_labels.append(labels[i].item())  
            filtered_scores.append(scores[i].item())  
  
    return filtered_boxes, filtered_labels, filtered_scores
```

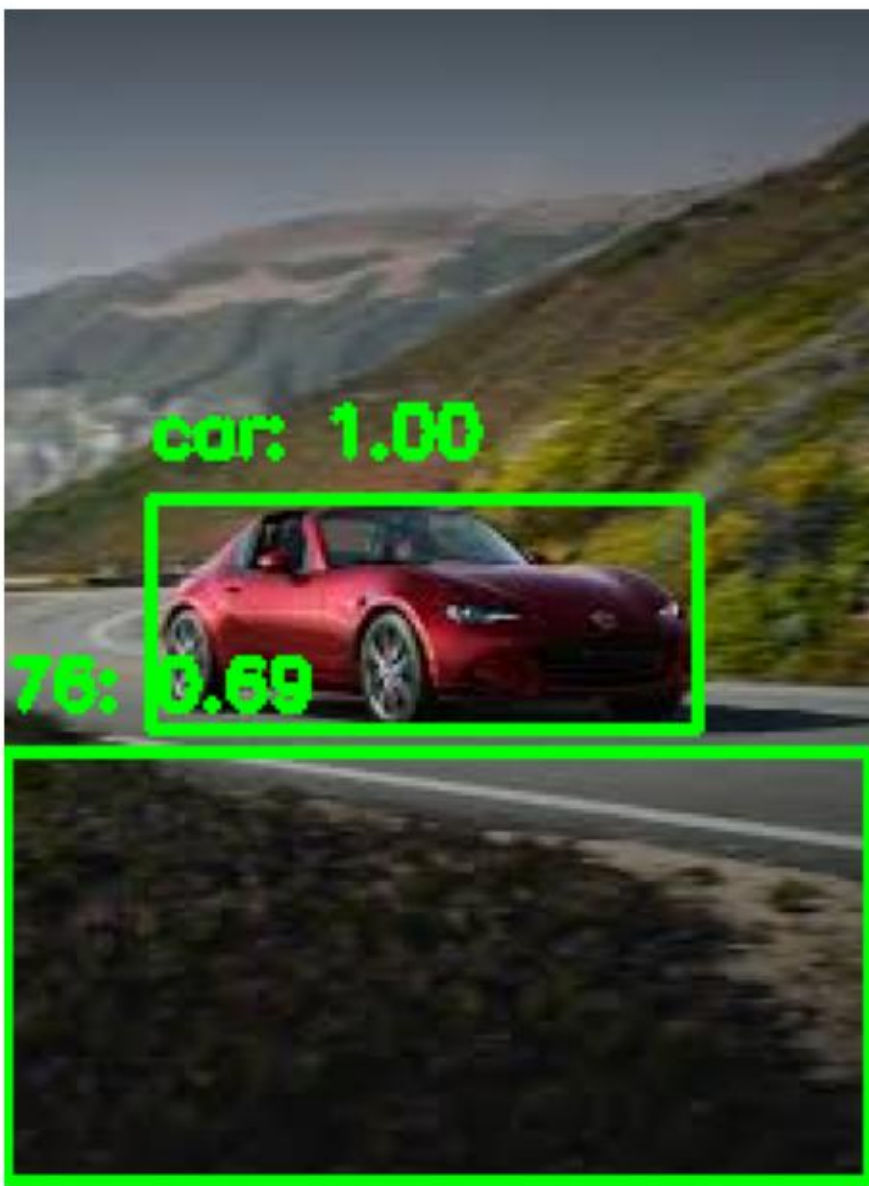
Visualization Function

```
[1] def draw_boxes(image, boxes, labels, scores):  
    image_np = np.array(image)  
  
    for box, label, score in zip(boxes, labels, scores):  
        x1, y1, x2, y2 = box.astype(int)  
  
        cv2.rectangle(image_np, (x1, y1), (x2, y2), (0,255,0), 2)  
  
        label_name = COCO_CLASSES[label] if label < len(COCO_CLASSES) else str(label)  
        text = f"{label_name}: {score:.2f}"  
  
        cv2.putText(image_np, text, (x1, y1-10),  
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,255,0), 2)  
  
    plt.figure(figsize=(10,8))  
    plt.imshow(image_np)  
    plt.axis("off")  
    plt.show()
```

[+ Code](#)[+ Text](#)

Test with our Image

```
1 image_path = "/content/car.jpeg" # upload your image in Colab  
image, image_tensor = load_image(image_path)  
boxes, labels, scores = detect_objects(model, image_tensor)  
draw_boxes(image, boxes, labels, scores)
```



▼ TASK 2: Use Two More Models (Different Backbones)

MobileNet Backbone (Lightweight)

```
[ ] ▶ model_mobilenet = torchvision.models.detection.fasterrcnn_mobilenet_v3_large_fpn(pretrained=True)
      model_mobilenet.eval()

      boxes, labels, scores = detect_objects(model_mobilenet, image_tensor)
      draw_boxes(image, boxes, labels, scores)
```

▼ ... /usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated. warnings.warn(msg)

Downloading: "https://download.pytorch.org/models/fasterrcnn_mobilenet_v3_large_fpn-fb6a3cc7.pth" to /root/.cache/torch/hub/checkpoints/fasterrcnn_mobilenet_v3_large_fpn-fb6a3cc7.pth [100%] | 74.2M/74.2M [00:00<00:00, 130MB/s]



Faster R-CNN ResNet50 (v2 Improved)

```
[ ] model_v2 = torchvision.models.detection.fasterrcnn_resnet50_fpn_v2(pretrained=True)
    model_v2.eval()

    boxes, labels, scores = detect_objects(model_v2, image_tensor)
    draw_boxes(image, boxes, labels, scores)
```



TASK 3: mAP Calculation

+ Code + Text

Download Dataset

+ Code + Text

```
[ ] !pip install roboflow

from roboflow import Roboflow
rf = Roboflow(api_key="2yhE0XoVP8VjERSiNyYa")
project = rf.workspace("mubashar-workspace-nd2nw").project("wildfire-vxrin-sende")
version = project.version(1)
dataset = version.download("yolov11")

Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from roboflow) (2.0.2)
Collecting opencv-python-headless==4.10.0.84 (from roboflow)
  Downloading opencv_python_headless-4.10.0.84-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (20 kB)
Requirement already satisfied: Pillow>=7.1.2 in /usr/local/lib/python3.12/dist-packages (from roboflow) (11.3.0)
Collecting pi-heif<2 (from roboflow)
  Downloading pi_heif-1.3.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.3 kB)
Collecting pillow-avif-plugin<2 (from roboflow)
  Downloading pillow_avif_plugin-1.5.5-cp312-cp312-manylinux_2_28_x86_64.whl.metadata (2.2 kB)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.12/dist-packages (from roboflow) (2.9.0.post0)
Successfully uninstalled opencv-python-headless-4.10.0.84
Attempting uninstall: idna
  Found existing installation: idna 3.11
  Uninstalling idna-3.11:
    Successfully uninstalled idna-3.11
Successfully installed filetype-1.2.0 idna-3.7 opencv-python-headless-4.10.0.84 pi-heif-1.3.0 pillow-avif-plugin-1.5.5 roboflow-1.3.1
loading Roboflow workspace...
loading Roboflow project...
Downloading Dataset Version Zip in wildfire-1 to yolov11:: 100% [115672/115672 [00:01<00:00, 65544.52it/s]]
Extracting Dataset Version Zip to wildfire-1 in yolov11:: 100% [1439/1439 [00:00<00:00, 3818.97it/s]]
```

Install Required Libraries

```
[ ] !pip install torchmetrics

Collecting torchmetrics
  Downloading torchmetrics-1.9.0-py3-none-any.whl.metadata (23 kB)
Requirement already satisfied: numpy>1.20.0 in /usr/local/lib/python3.12/dist-packages (from torchmetrics) (2.0.2)
Requirement already satisfied: packaging>17.1 in /usr/local/lib/python3.12/dist-packages (from torchmetrics) (26.0)
Requirement already satisfied: torch>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from torchmetrics) (2.10.0+cu128)
Collecting lightning-utilities>=0.15.3 (from torchmetrics)
  Downloading lightning_utilities-0.15.3-py3-none-any.whl.metadata (5.5 kB)
Requirement already satisfied: typing_extensions in /usr/local/lib/python3.12/dist-packages (from lightning-utilities>=0.15.3->torchmetrics) (4.15.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->torchmetrics) (3.25.2)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->torchmetrics) (75.2.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=2.0.0->torchmetrics) (3.0.3)
Downloading torchmetrics-1.9.0-py3-none-any.whl (983 kB)
983.4/983.4 kB 26.4 MB/s eta 0:00:00
Downloading lightning_utilities-0.15.3-py3-none-any.whl (31 kB)
Installing collected packages: lightning-utilities, torchmetrics
Successfully installed lightning-utilities-0.15.3 torchmetrics-1.9.0
```

Load Model

```
[ ] import torch
import torchvision

model = torchvision.models.detection.fasterrcnn_resnet50_fpn(pretrained=True)
model.eval()

The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.
Arguments other than a weight enum or 'None' for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing 'weights-FasterRCNN'.
(transform): GeneralizedRCNNTransform(
  Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
  Resize(min_size=(800,), max_size=1333, mode='bilinear')
)
```

Load Dataset (YOLO → PyTorch format)

```
[ ] ▶ import os
import cv2
import numpy as np

def load_yolo_annotations(label_path, img_width, img_height):
    boxes = []

    with open(label_path, 'r') as f:
        lines = f.readlines()

    for line in lines:
        cls, x, y, w, h = map(float, line.strip().split())

        # Convert YOLO → Pascal VOC
        x1 = (x - w/2) * img_width
        y1 = (y - h/2) * img_height
        x2 = (x + w/2) * img_width
        y2 = (y + h/2) * img_height

        boxes.append([x1, y1, x2, y2])

    return boxes
```

IoU Function

[+ Code](#)[+ Text](#)

```
[ ] ▶ def compute_iou(box1, box2):
    x1 = max(box1[0], box2[0])
    y1 = max(box1[1], box2[1])
    x2 = min(box1[2], box2[2])
    y2 = min(box1[3], box2[3])

    inter = max(0, x2-x1) * max(0, y2-y1)

    area1 = (box1[2]-box1[0]) * (box1[3]-box1[1])
    area2 = (box2[2]-box2[0]) * (box2[3]-box2[1])

    union = area1 + area2 - inter

    return inter / union if union > 0 else 0
```

mAP Calculation

```
[ ] ▶ def compute_ap(pred_boxes, gt_boxes, iou_threshold=0.5):
    matched = []
    TP = 0
    FP = 0

    for pred in pred_boxes:
        found_match = False

        for i, gt in enumerate(gt_boxes):
            if i in matched:
                continue

            if compute_iou(pred, gt) >= iou_threshold:
                TP += 1
                matched.append(i)
                found_match = True
                break

        if not found_match:
            FP += 1

    FN = len(gt_boxes) - len(matched)

    precision = TP / (TP + FP + 1e-6)
    recall = TP / (TP + FN + 1e-6)

    return precision, recall
```


Run on Dataset

+ Code

+ Text

```
1  from PIL import Image
    from torchvision import transforms

    transform = transforms.Compose([transforms.ToTensor()])

    image_dir = dataset.location + "/test/images"
    label_dir = dataset.location + "/test/labels"

    image_files = os.listdir(image_dir)

    all_precisions = []
    all_recalls = []

    for img_file in image_files[:20]: # take 20 images (you can increase)

        img_path = os.path.join(image_dir, img_file)
        label_path = os.path.join(label_dir, img_file.replace(".jpg", ".txt"))

        # Load image
        image = Image.open(img_path).convert("RGB")
        img_tensor = transform(image)

        # Get predictions
        with torch.no_grad():
            preds = model([img_tensor])[0]

        pred_boxes = preds['boxes'].numpy()
        scores = preds['scores'].numpy()

        # Filter predictions
        pred_boxes = [box for box, s in zip(pred_boxes, scores) if s > 0.5]

        # Load ground truth
        img_w, img_h = image.size
        gt_boxes = load_yolo_annotations(label_path, img_w, img_h)

        # Compute metrics
        precision, recall = compute_ap(pred_boxes, gt_boxes)

        all_precisions.append(precision)
        all_recalls.append(recall)

    # Final mAP (mean precision)
    mAP = np.mean(all_precisions)

    print("Final mAP:", mAP)
    print("Average Recall:", np.mean(all_recalls))

... Final mAP: 0.01999999600000008
   Average Recall: 0.0499999750000125
```

